

Hardware Access on Raspberry Pi with Linux

1 Accessing Hardware on Raspberry Pi with Linux

Even with Linux running on the Raspberry Pi, you can still access hardware peripherals without switching to bare-metal programming. Here are several ways to interact with the hardware while using Linux as the operating system:

1. Memory-Mapped I/O via `/dev/mem`

- Description: You can access hardware registers via memory-mapped I/O using the `/dev/mem` interface. This allows direct access to physical memory, enabling control of peripherals like GPIO, timers, and UART.
- Usage: Open `/dev/mem` with proper permissions, map the peripheral's physical address using `mmap()`, and interact with the hardware registers.

2. Using Device Files (e.g., `/dev/gpio`)

- Description: Linux provides device files under `/dev` for accessing peripherals such as GPIO, SPI, I2C, and UART. You can read from and write to these device files to interact with the hardware.
- Usage: Use system calls like `open()`, `read()`, and `write()` to access the peripheral.

3. GPIO Control with `sysfs` or `libgpiod`

- Description: The GPIO pins can be controlled via the `sysfs` interface, or using the `libgpiod` library. This allows user-space programs to control GPIO pins.
- Usage: Write to `sysfs` files (e.g., `/sys/class/gpio/export`, `/sys/class/gpio/gpioXX/value`) or use the `libgpiod` API to control GPIO pins.

4. Accessing UART, SPI, I2C via Linux Drivers

- Description: Linux provides drivers for UART, SPI, and I2C, making it easy to communicate with these peripherals via device files (e.g., `/dev/serial0`, `/dev/spidev0.0`).

- Usage: Open the device file and communicate using standard file I/O system calls.

5. Real-Time Programming with Preempt-RT Patch

- Description: If precise timing is required, the Preempt-RT patch for the Linux kernel reduces interrupt latency, making Linux behave closer to a real-time operating system.
- Usage: Compile Linux with the Preempt-RT patch and assign real-time priorities to tasks.

2 Bare-Metal Programming vs. Linux Programming

- **Bare-Metal Programming:** Provides full control over the hardware but sacrifices the conveniences of an operating system (e.g., multitasking, file systems, networking).
- **Linux Programming:** Allows access to peripherals and low-level hardware while still benefiting from OS-level features like multitasking and hardware abstraction.

3 Example: Accessing GPIO with /dev/mem

To access GPIO pins via assembly or C while running Linux:

1. Open `/dev/mem` using `open("/dev/mem", O_RDWR)`.
2. Map the GPIO registers using `mmap()` to a pointer in user space.
3. Control the GPIO pins by writing to the mapped memory addresses.